

Hashing and Hash Tables

Where We Left Off

Finding one item among n :

Unsorted array, linear search

$O(n)$

Sorted array, binary search

$O(\log n)$

Today we get to

$O(1)$ *average*

Constant time - independent of n

The Problem

*Our sensor readings have **names**, not numbers:*

<i>"oil_pressure"</i>	<i>1456 counts</i>
<i>"coolant_temp"</i>	<i>2011 counts</i>
<i>"fuel_level"</i>	<i>903 counts</i>

Given the name, find the reading

*But an array index is a **number***

Direct Addressing

Sensor ID is already a small integer:

readings [3] is the reading for sensor 3



902	1456	2011	77	1180	640	318	2044
0	1	2	3	4	5	6	7

No hashing needed - the key IS the index, 0 (1)

Why Direct Addressing Fails

Keys are usually not small integers:

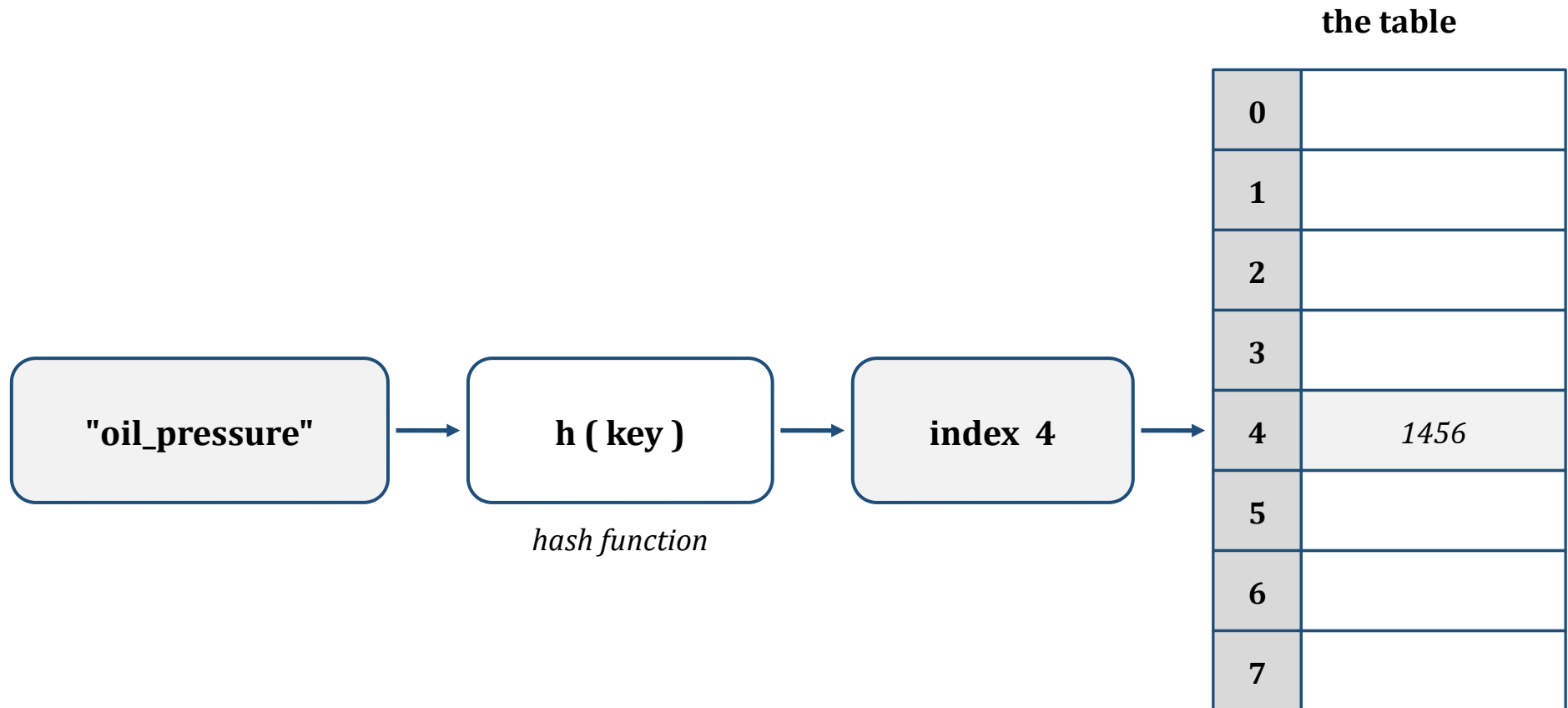
Strings *"oil_pressure" is not an index*

Sparse *sensor IDs 1001, 5544, 90210
three entries, 90,211 slots*

Wide *a 32-bit ID needs 4 billion slots*

*We need a way to **shrink** a key to an index*

The Hashing Idea



*We do not search for the slot - we **compute** it*

What a Hash Function Is

A function that maps a key to an integer:

$$h : \text{key} \rightarrow 0 .. m-1$$

*m is the number of **buckets** in the table*

*Same key $\rightarrow$ **always** the same bucket*

*Different keys $\rightarrow$ **usually** different buckets*

Properties of a Good Hash Function

Deterministic *same input, same output*

Fast *cheaper than the search it replaces*

Uniform *spreads keys evenly over m*

Avalanche *one bit in changes many bits out*

*It is **not** encryption and **not** a checksum*

The Division Method

The simplest useful hash:

$$\text{index} = k \% m$$

*k is the key as an integer,
m is the table size*

One instruction, always in range

*Everything depends on the choice of **m***

Why m Should Be Prime

*If $m = 16$, then $k \% 16$ keeps only the low **4 bits** - the rest of the key is thrown away*

$0x1234 \rightarrow 4$

$0xABC4 \rightarrow 4$

$0x0004 \rightarrow 4$

*A **prime** m makes every bit of k matter*

Hashing Strings: First Attempt

Add up the characters:

```
unsigned h = 0 ;  
while ( *s ) h += *s++ ;
```

"ab" → 97 + 98 = 195

"ba" → 98 + 97 = 195

*Every **anagram** collides, and the range is tiny*

djb2: A Better String Hash

```
unsigned long hash ( const char *s ) {  
    unsigned long h = 5381 ;  
    while ( *s )  
        h = h * 33 + ( unsigned char ) *s++ ;  
    return h ;  
}
```

*Multiplying **carries position** into the value
now "ab" and "ba" differ*

FNV-1a: Another Good Choice

```
unsigned hash ( const char *s ) {  
    unsigned h = 2166136261u ;  
    while ( *s ) {  
        h ^= ( unsigned char ) *s++ ;  
        h *= 16777619u ;  
    }  
    return h ;  
}
```

XOR then multiply - better avalanche than djb2

Hashing a Struct

*Hash exactly the fields you **compare***

```
h = hash ( s.adc_counts ) ;  
h = h * 33 + hash ( s.timestamp_ms ) ;
```

*Never hash the raw bytes of a struct:
padding bytes are uninitialized*

Two equal structs would hash differently

The Golden Rule

If $a == b$

then $h(a) == h(b)$

*Equality and hashing must **agree***

Break this and lookups silently fail

Collisions Are Unavoidable

Keys: billions of possible strings

Buckets: maybe 101

Pigeonhole principle

more pigeons than holes

→ some hole has two pigeons

*A collision is **not** a bug*

Handling it badly is

Worse Than You Expect

The birthday paradox

23 people in a room:

*about a **50%** chance two share a birthday*

Put $\sqrt{m}$ keys in m buckets:

a collision is already likely

365 buckets, 23 keys - plan for collisions

Two Ways to Handle Collisions

Separate chaining

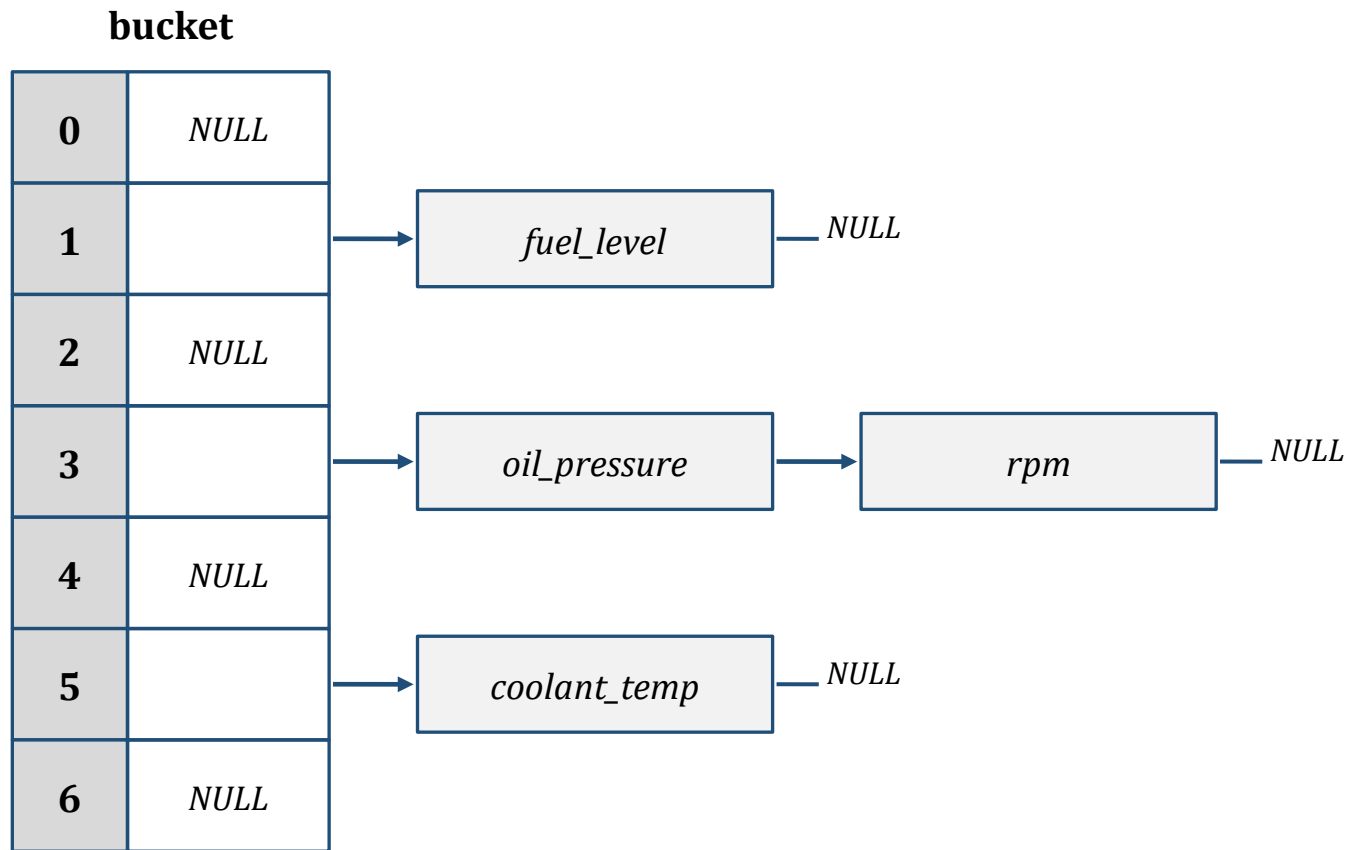
*each bucket holds a **list** of entries*

Open addressing

*one entry per bucket; if full,
go look in another bucket*

Every real hash table picks one of these

Separate Chaining



Each bucket holds a **list** of every key that hashed there

Chaining in C: The Entry

```
#define TABLE_SIZE    101
```

```
typedef struct entry {  
    char      *key;  
    int       value;  
    struct entry *next;  
} entry_t;
```

```
entry_t *tableG [ TABLE_SIZE ] = { NULL };
```

Chaining in C: Insert

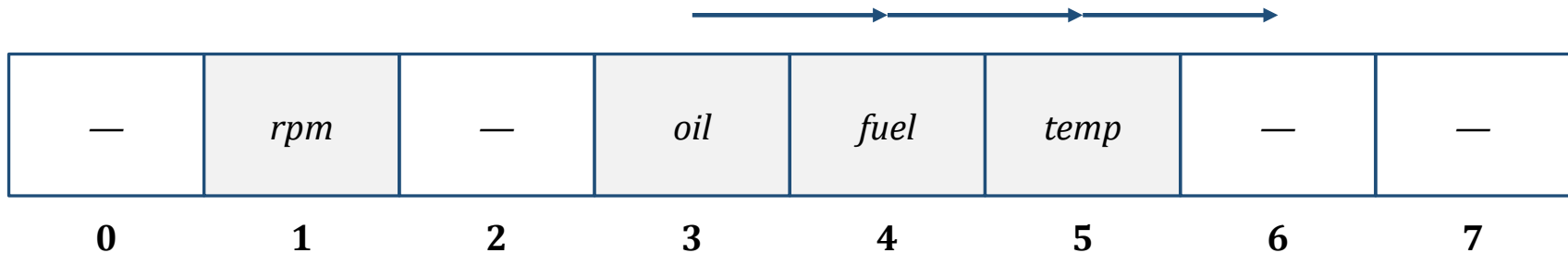
```
void insert ( const char *key, int value ) {  
    unsigned idx = hash ( key ) % TABLE_SIZE ;  
  
    entry_t *e = malloc ( sizeof ( entry_t ) ) ;  
    e->key      = strdup ( key ) ;  
    e->value    = value ;  
    e->next     = tableG [ idx ] ;  
    tableG [ idx ] = e ;  
}
```

Chaining in C: Lookup

```
int *lookup ( const char *key ) {  
    unsigned idx = hash ( key ) % TABLE_SIZE ;  
  
    for ( entry_t *e = tableG [ idx ] ; e ; e = e->next )  
        if ( strcmp ( e->key, key ) == 0 )  
            return &( e->value ) ;  
  
    return NULL ;                                /* not found */  
}
```

Open Addressing: Linear Probing

$h(\text{"speed"}) = 3$, but slot 3 is taken



*Try 3, 4, 5, 6 ... take the **first empty slot***

Lookup repeats the same walk and stops at the first empty slot

Linear Probing in Code

$$unsigned\ idx = hash (key) \% m ;$$

```
while ( table [ idx ].used ) {
    if ( strcmp ( table [ idx ].key, key ) == 0 )
        return idx ;                               /* found */
    idx = ( idx + 1 ) % m ;                          /* next slot */
}
```

```
return NOT_FOUND;           /* hit an empty slot */
```

Primary Clustering

Linear probing makes runs of full slots

*A long run gets **longer**:
any key landing in it walks to the end*

Two clusters that touch become one big one

Cost grows much faster than n does

Quadratic and Double Hashing

Linear $idx + 1, idx + 2, idx + 3 \dots$

Quadratic $idx + 1, idx + 4, idx + 9 \dots$
breaks up clusters

Double $idx + 1 \cdot h2(k), idx + 2 \cdot h2(k) \dots$
each key gets its own step size

$h2$ must never return 0

Deletion and Tombstones

Open addressing cannot just empty a slot

*Lookup stops at the first **empty** slot,
so an emptied slot **hides** everything behind it*

*Mark it **DELETED** instead - a **tombstone**
probing walks past it
insert may reuse it*

Too many tombstones → rebuild the table

The Load Factor

$$\alpha = n / m$$

n = entries stored

m = buckets available

$\alpha = 0.5$ *half full*

$\alpha = 1.0$ *as many entries as buckets*

α is the one number that predicts speed

Cost of Chaining

Average list length = α

Successful lookup $\approx 1 + \alpha/2$ comparisons

Unsuccessful lookup $\approx 1 + \alpha$ comparisons

Both are $O(1 + \alpha)$

Keep α bounded and this is $O(1)$

Cost of Linear Probing

α	Chaining	Probing: hit	Probing: miss
0.25	1.25	1.2	1.4
0.50	1.50	1.5	2.5
0.75	1.75	2.5	8.5
0.90	1.90	5.5	50.5
0.95	1.95	10.5	200.5

Average slots touched per lookup

Linear probing falls apart as the table fills

The Practical Rule

Keep $\alpha \leq 0.7$

When the table passes that point:

*allocate a **bigger** table (usually $2\times$)*

***rehash** every entry into it*

free the old table

The new m changes every index

Rehashing Is $O(n)$ - Once

One rehash costs $O(n)$

But it doubles the capacity, so it happens less and less often:

grow at 8, 16, 32, 64, 128 ...

Total cost of n inserts = $O(n)$

Each insert is $O(1)$ amortized

The Worst Case Is $O(n)$

*If every key hashes to the **same** bucket:*

chaining $\rightarrow$ one long linked list

probing $\rightarrow$ one long walk

Lookup becomes $O(n)$

*A bad hash function turns a hash table
into a linked list with extra steps*

Hash Flooding

*An attacker who knows h can craft keys that **all** land in one bucket*

n requests $\rightarrow O(n^2)$ work $\rightarrow$ server stalls

*The defence: a **random seed** per process
same key hashes differently each run*

Rust does this by default; C++ does not

Complexity Summary

Operation	Average	Worst case
Insert	$O(1)$	$O(n)$
Lookup	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$
Visit every entry	$O(n + m)$	$O(n + m)$
Space	$O(n)$	$O(n)$

Average assumes a good hash and α kept small

Compared With What We Know

Structure	Lookup	Sorted order	Extra memory
Unsorted array	$O(n)$	no	none
Sorted array	$O(\log n)$	yes	none
Balanced tree	$O(\log n)$	yes	pointers
Hash table	$O(1)$ avg	no	empty buckets

A hash table trades **ordering** for **speed**

Space Complexity

A hash table stores n entries in m buckets

$m > n$ always (we keep $\alpha \leq 0.7$)

Chaining	<i>n entries + m pointers + n next pointers</i>
Probing	<i>m slots, about 30% of them empty</i>

Still $O(n)$ - but with a real constant

Hash Tables in C

*The C standard library has **no** hash table*

Your choices:

write one (we just did)

uthash header-only macros

GLib GHashTable

POSIX hcreate / hsearch (one table only)

This is why C programmers know how these work

uthash in C

```
#include "uthash.h"
```

```
typedef struct reading {  
    char          name [ 32 ];          /* the key */  
    int           counts ;  
    UT_hash_handle hh ;                /* bookkeeping */  
} reading_t ;
```

```
reading_t *readingsG = NULL ;  
HASH_ADD_STR ( readingsG, name, r ) ;  
HASH_FIND_STR ( readingsG, "oil_pressure", r ) ;
```

C++: `std::unordered_map`

```
#include <unordered_map>  
#include <string>
```

```
std::unordered_map < std::string, int > readings ;
```

```
readings [ "oil_pressure" ] = 1456 ;  
readings [ "coolant_temp" ] = 2011 ;
```

```
int c = readings [ "oil_pressure" ] ;
```

C++: The [] Trap

if (readings ["rpm"] == 0) ...

*This **inserts** "rpm" with value 0*

To look without inserting:

```
if ( readings.count ( "rpm" ) ) ...  
auto it = readings.find ( "rpm" );  
if ( it != readings.end ( ) ) use ( it->second );
```

C++20: if (readings.contains ("rpm")) ...

C++: Iterating and Erasing

```
for ( const auto & [ name, counts ] : readings )  
    std::cout << name << " = " << counts << "\n";
```

```
readings.erase ( "fuel_level" );
```

```
readings.reserve ( 1000 );           /* avoid rehashing */
```

```
readings.load_factor ( );           /* current  $\alpha$  */  
readings.max_load_factor ( 0.75f );
```

C++: `std::unordered_set`

```
#include <unordered_set>
```

```
std::unordered_set < int > seen ;
```

```
if ( ! seen.insert ( id ).second )  
    std::cout << "duplicate id\n" ;
```

A map with keys and no values

This is the $O(n)$ duplicate finder from last lecture

C++: map vs unordered_map

	std::map	std::unordered_map
Structure	<i>balanced tree</i>	<i>hash table</i>
Lookup	<i>$O(\log n)$</i>	<i>$O(1)$ average</i>
Order	<i>sorted by key</i>	<i>unspecified</i>
Key needs	<i>operator <</i>	<i>std::hash + ==</i>
Rust name	<i>BTreeMap</i>	<i>HashMap</i>

Need keys in order? Use the tree.

C++: Hashing Your Own Type

```
struct key_t { int sensor ; int channel ; } ;
```

```
template < > struct std::hash < key_t > {  
    size_t operator ( ) ( const key_t & k ) const {  
        return std::hash<int>{} ( k.sensor ) * 33  
            ^ std::hash<int>{} ( k.channel ) ;  
    }  
};
```

You must also supply **operator ==**

Rust: HashMap

```
use std::collections::HashMap ;  
  
let mut readings : HashMap < String, u16 > = HashMap::new ( ) ;  
  
readings.insert ( "oil_pressure".to_string ( ), 1456 ) ;  
  
if let Some ( c ) = readings.get ( "oil_pressure" ) {  
    println! ( "{} counts", c ) ;  
}
```

Rust: The entry API

Count how many times each command arrives:

```
let mut counts : HashMap < & str, u32 > = HashMap::new ( ) ;  
  
for cmd in log.iter ( ) {  
    *counts.entry ( cmd ).or_insert ( 0 ) += 1 ;  
}
```

Rust: Other entry Forms

map.entry (k).or_insert_with (Vec::new);

map.entry (k).or_default ();

*map.entry (k).and_modify (|v| *v += 1)
 .or_insert (1);*

*or_insert_with builds the value **only if needed***

Rust: HashSet

```
use std::collections::HashSet;
```

```
let mut seen : HashSet < u32 > = HashSet::new ();
```

```
if ! seen.insert ( id ) {  
    println! ( "duplicate {}", id );  
}
```

*insert returns **true** if the value was new*

Rust: Deriving Hash

```
#[ derive ( Hash, PartialEq, Eq ) ]  
struct Key {  
    sensor : u8,  
    channel : u8,  
}
```

*Now Key can be a **key***

*The compiler writes both, and they **agree***

Rust: Why f32 Cannot Be a Key

*f32 and f64 do **not** implement Eq*

NaN != NaN

A key that is not equal to itself can never be found again

*Store the **raw counts**, not the psi*
u16 is a key; f32 is not

Rust: Iteration Order

```
for ( name, counts ) in &readings {  
    println! ( "{}: {}", name, counts );  
}
```

*The order is **not defined**
and changes between runs*

*Need order? use **BTreeMap**
or collect and sort*

Rust: Choosing a Hasher

*Default: **SipHash 1-3** with a random seed
safe against hash flooding
not the fastest*

*For trusted, internal keys:
use `rustc_hash::FxHashMap` ;
use `ahash::AHashMap` ;*

Same API, different speed/safety trade

The Three Languages

	C	C++	Rust
Hash map	<i>none</i>	<i>unordered_map</i>	<i>HashMap</i>
Hash set	<i>none</i>	<i>unordered_set</i>	<i>HashSet</i>
Collisions	<i>you choose</i>	<i>chaining</i>	<i>open addressing</i>
Default hash	<i>you write it</i>	<i>std::hash</i>	<i>SipHash 1-3</i>
Random seed	<i>no</i>	<i>no</i>	<i>yes</i>
Ordered map	<i>none</i>	<i>std::map</i>	<i>BTreeMap</i>
Perfect hash	<i>gperf</i>	<i>gperf / frozen</i>	<i>phf crate</i>

C gives you nothing - which is why we built one

Perfect Hashing

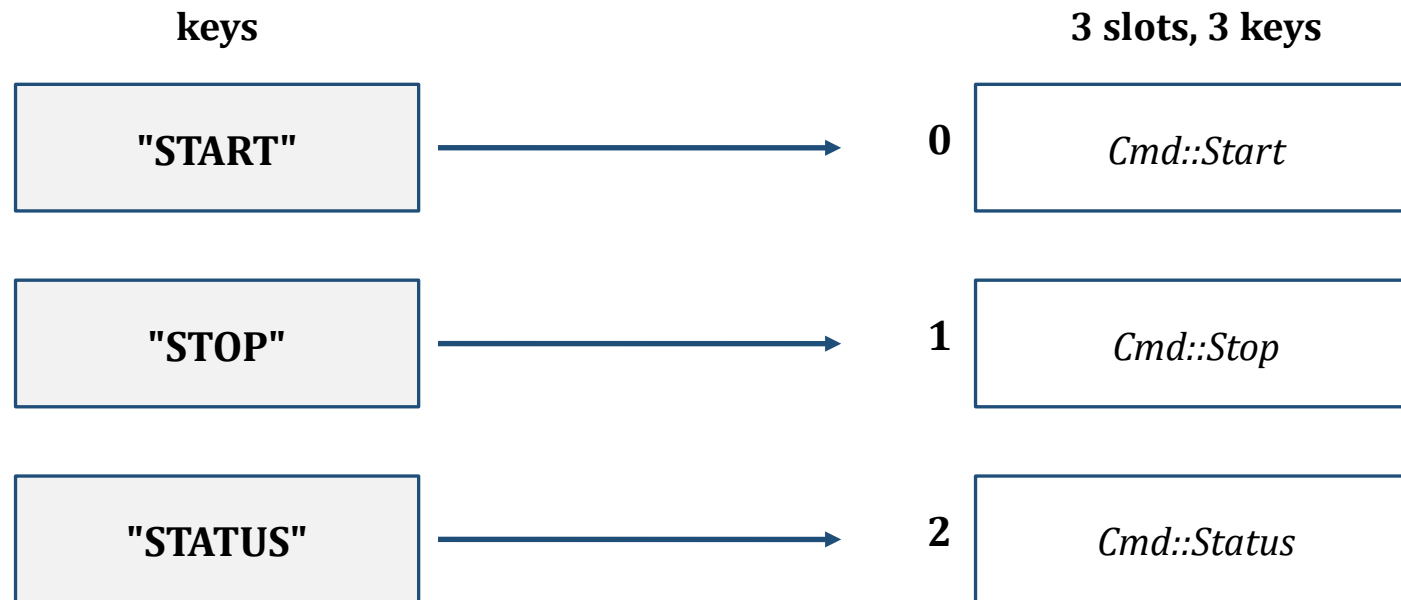
*What if the set of keys is **known in advance**?*

*Reserved words, opcodes, command names,
HTTP headers, CSS properties*

*Then we can build a hash function with
no collisions at all*

*Lookup becomes **$O(1)$** worst case*

Perfect and Minimal



No collisions. *Three keys, three slots - minimal perfect hashing*
 $O(1)$ worst case, not just average

The Two Conditions

Perfect *no two keys collide*
 $m \geq n$

Minimal perfect *perfect, and $m = n$*
 every slot used, $\alpha = 1.0$

*Both require the key set to be **fixed***

Add one key $\rightarrow$ build a new function

Where Perfect Hashing Is Used

Compilers *is this word a keyword?*

Assemblers *instruction mnemonics*

Web servers *HTTP header names*

Firmware *command tables in flash*

Small fixed sets, looked up constantly

gperf: Generating a Perfect Hash

Our three commands from the enums lecture:

```
%{  
#include "commands.h"  
%}  
%%  
START,          CMD_START  
STOP,           CMD_STOP  
STATUS,         CMD_STATUS  
%%
```

gperf: What It Generates

```
$ gperf -t commands.gperf > commands.c
```

```
static unsigned hash ( const char *s, size_t len )  
{  
    static const unsigned char asso [ ] = { ... } ;  
    return len + asso [ s [ 0 ] ] + asso [ s [ len-1 ] ] ;  
}
```

*Two table lookups and two adds - **no loop***

gperf: Using the Result

```
const cmd_t *c = in_word_set ( buf, strlen ( buf ) ) ;
```

```
if ( c != NULL )  
    dispatch ( c->code ) ;
```

```
else  
    report ( ERR_UNKNOWN_COMMAND ) ;
```

*You **still** compare the string once
an unknown word can hash to a valid slot*

FKS: Perfect Hashing in Theory

Two levels:

Level 1 *hash n keys into n buckets*

Level 2 *bucket with k keys gets its own
table of k^2 slots*

k^2 slots $\rightarrow$ a collision-free h is easy to find

*Total space is still **$O(n)$***

Rust: the phf crate

```
use phf::phf_map ;
```

```
static CMDS : phf::Map < & 'static str, Cmd > = phf_map! {  
    "START"      => Cmd::Start,  
    "STOP"       => Cmd::Stop,  
    "STATUS"     => Cmd::Status,  
};
```

*Built at **compile time**, stored in **flash***

C++: Perfect Hashing

No standard tool, but the same options:

gperf	<i>works on C++ too</i>
frozen	<i>constexpr maps, header-only</i>
constexpr	<i>build your own at compile time</i>

```
constexpr frozen::unordered_map < ... > cmds = {  
    { "START", Cmd::Start }, ... };
```

Perfect vs Ordinary

	Ordinary hash	Perfect hash
Key set	<i>unknown, grows</i>	<i>fixed, known</i>
Collisions	<i>expected</i>	<i>none</i>
Lookup	<i>$O(1)$ average</i>	<i>$O(1)$ worst case</i>
Built	<i>at run time</i>	<i>at build time</i>
Add a key	<i>just insert</i>	<i>regenerate</i>
Memory	<i>$\alpha \leq 0.7$, 30% spare</i>	<i>exactly n slots</i>

*Use a perfect hash when the keys ship **with the program***

Worked Example

A log of commands arrives from the ESP32:

STATUS START STATUS READ START STATUS

Count how many times each one appeared

*How would you do it with an **array**?*

Example in C++

```
std::unordered_map < std::string, int > counts ;
```

```
for ( const std::string & cmd : log )  
    counts [ cmd ] ++ ;
```

```
for ( const auto & [ cmd, n ] : counts )  
    std::cout << cmd << " " << n << "\n" ;
```

Here [] inserting 0 is exactly what we want

Example in Rust

```
let mut counts : HashMap < & str, u32 > = HashMap::new ( ) ;
```

```
for cmd in log.iter ( ) {  
    *counts.entry ( cmd ).or_insert ( 0 ) += 1 ;  
}
```

```
for ( cmd, n ) in &counts {  
    println! ( "{} {}", cmd, n ) ;  
}
```

Example in C

```
for ( int i = 0 ; i < n_log ; i++ ) {  
    int *p = lookup ( log [ i ] ) ;  
    if ( p )                *p += 1 ;  
    else                    insert ( log [ i ], 1 ) ;  
}
```

Our own table from earlier

Two lines in C++, seven in C, plus the table

Complexity of the Example

Array scan $O(n \cdot k)$
n log entries, k known commands

Hash table **$O(n)$** *average*

Sort then count $O(n \log n)$

For $n = 100,000$ and $k = 20$:
2 million compares vs 100,000

Seeing a Hash Table in the Debugger

Our C table is just an array of pointers:

```
(gdb) p tableG [ 42 ]
```

```
(gdb) p *tableG [ 42 ]
```

```
(gdb) p *tableG [ 42 ]->next
```

*Walk the chain and you can **see** the collision*

C++ and Rust:

```
p readings.size ( )
```

```
p readings
```

Hash Tables on a Microcontroller

Ask before you reach for one:

How many keys? *under 10 → a linear scan wins*

Fixed keys? *→ perfect hash in flash*

Is there a heap? *chaining calls malloc*

*An ESP32-S3 has RAM measured in **kilobytes***

Choosing a Structure

If you need ...	Use
<i>Key is a small integer</i>	<i>a plain array</i>
<i>Fewer than about 10 keys</i>	<i>a linear scan</i>
<i>Fixed keys, known at build</i>	<i>a perfect hash</i>
<i>Fast lookup by any key</i>	<i>a hash table</i>
<i>Keys in sorted order</i>	<i>a tree: map or BTreeMap</i>
<i>Ranges or nearest match</i>	<i>a tree</i>
<i>Membership only</i>	<i>a hash set</i>

Common Mistakes

*Changing a key **after** it is inserted
the entry is now in the wrong bucket*

*Hashing and comparing **different** fields*

Using `[]` in C++ to test for a key

Letting α reach 1.0 with open addressing

Assuming the iteration order means anything

What We Can Now Do

Compute *a bucket instead of searching*

Choose *a hash function and a table size*

Handle *collisions by chaining or probing*

Predict *cost from the load factor α*

Generate *a perfect hash for fixed keys*

Let's practice

*Exercise: Using **hashpractice***